



AULA 1

Bancos de dados: fundamentos, modelos, arquiteturas de sistemas e SQL básico



O PROFESSOR

Sérgio Lifschitz

Professor Associado do quadro principal do Departamento de Informática da PUC-Rio. Doutor em Bancos de Dados e Redes pela École Nationale Supérieure de Télécommunications (ENST Paris), França; Mestre e Bacharel em Engenharia Elétrica, ambos pela PUC-Rio. Coordenador do Laboratório BioBD de Bancos de Dados e Bioinformática e um dos coordenadores do curso de pós-graduação *lato sensu* em Análise e Projeto de Sistemas da CCE PUC-Rio.

Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Aprender os conceitos dos sistemas de bancos de dados.
- Entender os fundamentos das arquiteturas e modelos de dados.
- Compreender o modelo relacional, estruturas e restrições.
- Conhecer a linguagem SQL em suas expressões básicas.

Que história é essa?

Esta disciplina tem por objetivo ensinar ao aluno os fundamentos principais de bancos de dados, com equilíbrio na apresentação entre teoria e prática, nesta área tão relevante da computação e ciência de dados em geral. Serão vistos conceitos importantes para o desenvolvimento de sistemas de bancos de dados, desde os modelos de dados e arquiteturas operacionais, passando pela manipulação realizada pela linguagem SQL, pelo projeto e concepção de novos bancos de dados que respondem aos requisitos impostos pelas aplicações, até aspectos mais avançados ligados ao desempenho dos sistemas.

Esta primeira aula aborda a definição de sistemas de bancos de dados, os modelos de dados e suas características, em particular o **modelo de dados relacional**. O aluno é introduzido inicialmente à parte estrutural do modelo, as relações, para posteriormente conhecer uma fração relevante, voltada para consultas, da linguagem relacional SQL. Também enfatizaremos aqui as restrições de integridade que completam a especificação dos esquemas de bancos de dados, em particular os sistemas relacionais.

Para começarmos, vamos entender um pouco mais a arquitetura dos computadores.

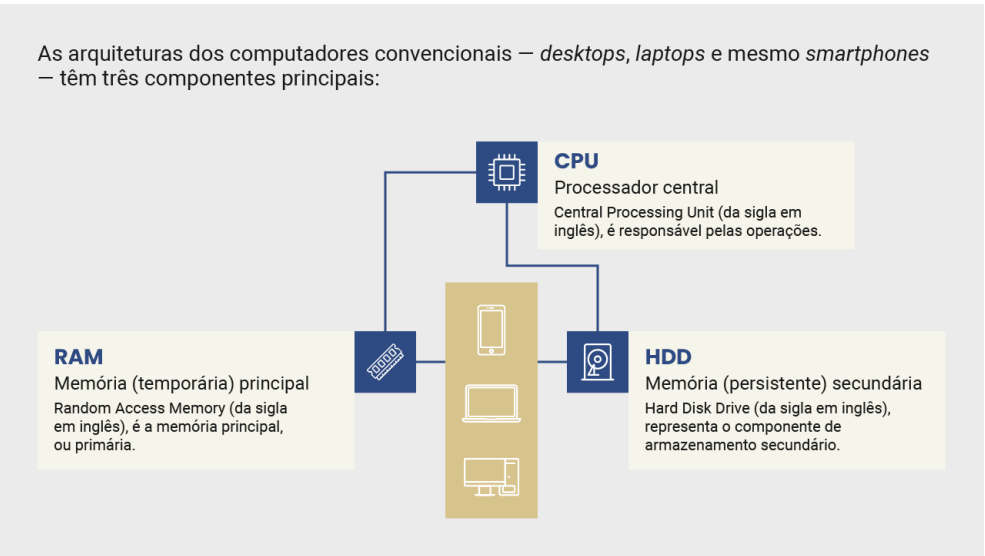


Figura 1.01 - Componentes básicos de um computador



Os termos **primário** e **secundário** são relevantes para quem usa bancos de dados. A CPU só acessa diretamente dados na RAM, que é bem menor, em bytes, do que um HDD. Como nem sempre os dados que precisamos cabem todos na RAM, o sistema operacional em uso deve buscá-los no HDD e substituir os dados presentes na RAM pelos dados recém-obtidos do HDD. No caso de grandes quantidades de dados, estes procedimentos precisam ser feitos

muitas vezes, e o custo em termos de tempo de processamento é alto, com consequências diretas no desempenho do sistema como um todo.

Os bancos de dados são relevantes para diversos sistemas e aplicações com impacto direto na vida das pessoas. São muitas as áreas científicas ou tecnológicas que utilizam dados para estudos, argumentações e análises diversas, já não se restringindo somente às ditas ciências exatas. Da medicina ao jornalismo, do direito às ciências sociais, não faltam exemplos do uso de grandes volumes de dados no dia a dia de diversos acadêmicos e profissionais.

Sistemas de bancos de dados

É verdade que muitas pessoas se referem aos bancos de dados como um “bando” de dados: um **conjunto de dados relativamente organizado e estruturado para satisfazer algumas demandas de aplicações específicas**. É o caso de aplicações contábeis que usam muitas planilhas de cálculo ou ainda quando nos referimos à gestão de coleções pessoais de livros ou fotografias. Os bancos de dados computacionais envolvem não apenas os dados em si, mas também, e principalmente, os **Sistemas Gerenciadores de Bancos de Dados (SGBD)**, que permitem administrar, consultar e atualizar os dados com segurança, eficácia e eficiência. Podemos, assim, chamar de sistemas de bancos de dados aquelas aplicações contemplando dados e algum SGBD.

A área de banco de dados se diferencia das demais por pelo menos dois aspectos:

1 – O fato de sempre lidar com grandes volumes de dados

2 – A necessidade de persistência destas grandes quantidades de dados

que exigem técnicas específicas de gestão e armazenamento, garantindo integridade e consistência dos dados.

É possível citar como exemplo típico o problema de ordenação de conjunto de valores por meio de programas computacionais. As técnicas e algoritmos mais conhecidos consideram que todos os dados cabem na memória principal, o que não é verdade para bancos de dados volumosos, também conhecidos pela sigla VLDB (*Very Large Data Bases*).

Aprenda mais

Para explicar o que são bancos de dados, é bom começar pelo conceito de modelo de dados, fundamental para a compreensão dos sistemas de bancos de dados existentes. Normalmente é dada ênfase ao modelo de dados relacional, padrão de fato e de mercado na maioria dos sistemas existentes. Entretanto, muitas das definições e termos aqui citados se aplicam aos modelos de dados em geral, inclusive os não relacionais.

Cabe observar que apenas os chamados modelos de dados lógicos, que contemplam tanto a estrutura quanto a forma de manuseio dos dados, são considerados modelos de dados por completo. É preciso contemplar as duas coisas: por exemplo, o **Modelo de Entidades e Relacionamentos (MER)**, proposto por Peter Chen em 1976 e que veremos em outro momento deste curso, é um modelo conceitual de dados, e contempla apenas a parte de representação da estrutura dos dados, sem oferecer mecanismos para seu manuseio. A figura abaixo traz um exemplo dos artefatos presentes no MER em sua representação diagramática. Temos duas entidades (Pessoas e Automóveis) e um relacionamento (Possuem), relacionando elementos (ou ocorrências) de pessoas e automóveis entre si. Os números 1 e N são cardinalidades (no caso, máximas) do relacionamento, que ajudam na compreensão dos conceitos presentes. Nesse caso, a leitura é “Pessoas podem ter mais de um (N) automóvel, mas cada automóvel pertence a apenas uma pessoa de cada vez”.

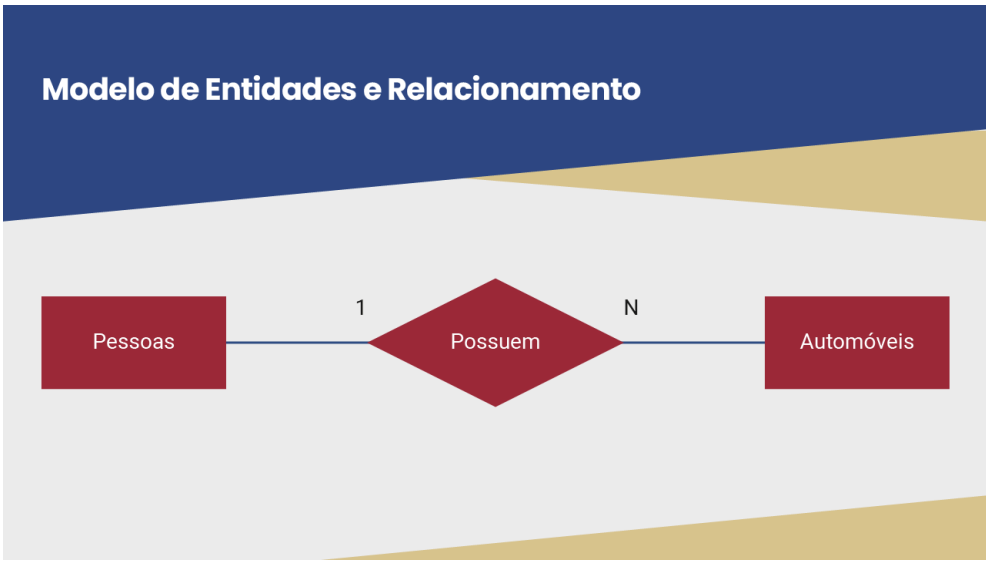


Figura 1.02 - Modelo de entidades e relacionamentos



Serão discutidos também os aspectos referentes às restrições de integridade, estruturais e semânticas, que complementam a especificação de qualquer sistema de banco de dados antes dele entrar em produção.

Além de oferecer uma definição formal, são mostrados alguns exemplos ilustrativos que podem ajudar na compreensão. O modelo de dados relacional, em particular, é formalmente definido e detalhado, dada sua importância na literatura, e também por ser a base da implementação de diversos sistemas computacionais utilizados em aplicações práticas diversas. A linguagem mais largamente utilizada, conhecida como SQL, será abordada em detalhes, com seus operadores básicos e alguns estendidos, permitindo que se conheça melhor como se pode consultar, criar e atualizar os dados num banco relacional.

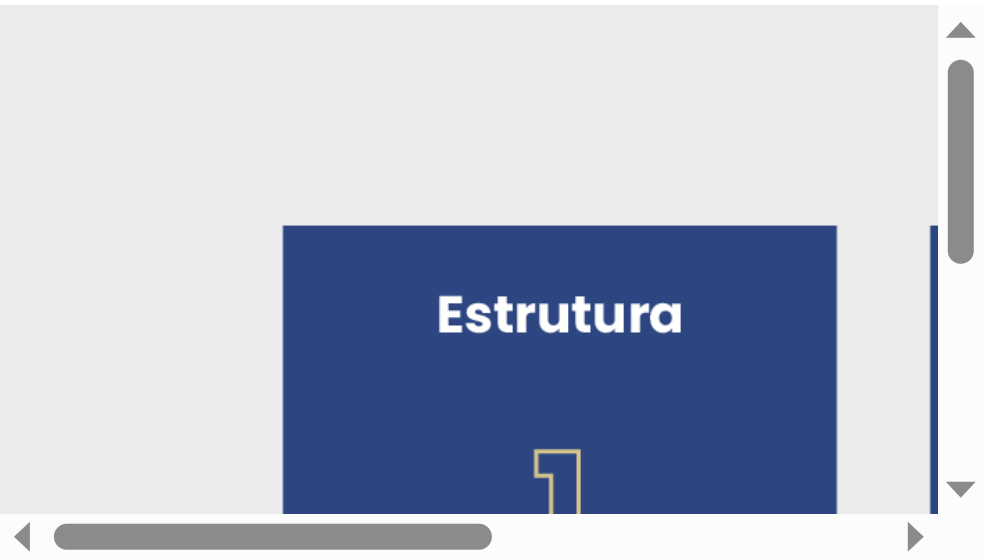
Modelos de dados

É fato que na literatura especializada sobre modelagem de sistemas de informação consideram-se vários níveis de abstração para os chamados modelos de dados (*data models*).

Pode-se partir de modelos mais distantes da implantação de um sistema, os chamados modelos de dados conceituais, até os modelos de dados de implementação, que podem incluir a escolha de um SGBD específico, e que têm a preocupação de representar os dados de tal maneira que incluem alguns requisitos de desempenho.

No caso específico da literatura de bancos de dados, o foco recai no que se costuma chamar de **modelos lógicos de dados** (*logical data models*), que podem ser efetivamente implementados em sistemas computacionais. Na maioria das vezes, quando se utiliza o termo modelo de dados, a referência é um modelo lógico de dados. Assim, exceto quando for necessária a distinção, será utilizado de agora em diante apenas o termo modelo de dados.

Podemos definir um modelo de dados como um formalismo matemático que contém duas partes:



Interativo

Dois quadrados de fundo azul com textos em branco. Na base de cada quadrado, um retângulo bordô, escrito: Saiba mais. No primeiro quadrado: Estrutura e o número 1. Ao clicar no Saiba mais, o texto: Formato de como os dados devem ser identificados, representados e abstraídos. No segundo quadrado: Manuseio e o número 2. Ao clicar no Saiba mais, o texto: Maneira como os dados são acessados ou atualizados por meio de operações definidas sobre reprodução estrutural.

Cabe observar que a maior parte das referências bibliográficas usa o termo “manipulação” e não “manuseio” para a 2ª parte do formalismo, pois esta é a tradução mais imediata da palavra *manipulation*, em inglês. Entretanto, como na língua portuguesa o ato de “manipular dados” é muitas vezes interpretado de maneira negativa ou pejorativa, será dada preferência ao termo “manuseio”.

Considera-se também que faz parte de um modelo de dados a definição de um **conjunto de restrições de integridade**, as quais, na prática, determinam o que pode e o que não pode fazer parte do conjunto de dados da base em questão. Há restrições ditas estruturais, que são dependentes da forma como os dados são representados e, assim, ligadas diretamente ao modelo de dados, e outras restrições chamadas de semânticas, pois são associadas às regras de negócio (*business rules*) da aplicação ou sistema que se quer implantar.

Modelo de dados relacional

Trata-se de um modelo de dados especificado e proposto por Edgar (Ted) F. Codd em 1970. Abaixo você encontra os artigos seminais dos professores e pesquisadores Ted Codd, sobre o modelo relacional, e Peter Chen, sobre a modelagem de entidades e relacionamentos. Em particular, Codd, mais de dez anos depois (1981), foi agraciado com o Prêmio Turing de computação exatamente por esta sua invenção, que influenciou toda uma nova geração de sistemas computacionais. O Prêmio Turing é considerado equivalente ao conhecido Prêmio Nobel, que é ofertado em outras áreas da ciência.

Aprenda mais

Para maior compreensão do tema, recomendamos a leitura dos artigos seminais dos autores Edgar (Ted) F. Codd e Peter Chen (em inglês):

1 - **CODD, E.F., “A Relational Model of Data for Large Shared Data Banks”**. [Clique aqui](#) e acesse!

2 - **CHEN, P.P.S., “The Entity-Relationship Model-Toward a Unified View of Data”**. [Clique aqui](#) e acesse!



É importante considerar o contexto em que este modelo para bancos de dados foi proposto: no final dos anos 60 havia poucos computadores, todos similares aos chamados *mainframes*, e o uso deles dependia de especialistas ainda formados em cursos de base matemática, como física ou engenharia. Os sistemas de informação contendo bancos de dados tinham seus desempenhos bastante dependentes da qualidade dos programadores que os haviam desenvolvido. As redes de computadores, como a internet, ainda mal apareciam nos sonhos dos profissionais da época. Muitos autores ativos na área, e que vivenciaram o nascimento do modelo relacional, mencionam que os sistemas então existentes eram desorganizados e *ad hoc*, com o esforço de desenvolvimento raramente reutilizado de um programa para outro.

O modelo relacional (*relational data model*) contempla as duas partes que compõem qualquer modelo de dados: **estrutura**, no caso relações matemáticas, e o **manuseio** por meio de linguagens (por exemplo, SQL ou álgebra relacional) projetadas para estruturas relacionais.

É importante apontar logo para uma interpretação errada que se dá ao termo “relacional” deste modelo lógico de dados. Certamente não

diz respeito ao fato de que representaria os relacionamentos sobre os dados do banco, pois, na verdade, todo modelo de alguma maneira especifica a forma como os dados se relacionam. Na verdade, o modelo foi assim nomeado pois **sua estrutura se baseia em abstrações matemáticas chamadas de relações**. Além disso, acredita-se que esta incompreensão quanto ao nome do modelo também seja motivada pela ubiquidade do modelo conceitual de entidades e relacionamentos, proposto depois do modelo relacional, e pela falta de distinção que muitos fazem entre os termos relação (*relation*) e relacionamento (*relationship*).

Estrutura do modelo de dados relacional

A parte estrutural do modelo relacional é baseada no conceito matemático de relações, que tem, por sua vez, muitas semelhanças com o conceito matemático de conjunto. Como será visto, muitas das operações sobre relações são oriundas da teoria de conjuntos, e por isso mesmo o modelo é dito **orientado aos conjuntos (set-oriented)**.

Uma relação pode ser enxergada ou representada como uma tabela com linhas e colunas, cujos elementos são as linhas. Alternativamente, pode-se visualizar uma relação como um conjunto de colunas, quando o foco se dá nos domínios das colunas, ou atributos representados. Mais formalmente, um elemento de uma relação é chamado de **tupla**, que é representada como uma linha de uma tabela. Uma tupla é um **conjunto não ordenado de valores, um para cada coluna**. Cada valor é derivado do seu domínio apropriado. Uma relação pode ser vista como um conjunto de tuplas, ou um conjunto de linhas. Colunas numa tabela são, formalmente, **atributos** nas relações. Para facilitar a compreensão da noção das tuplas ou t-uplas, se fossem duas ou três colunas poderíamos dizer duplas ou triplas, respectivamente.

Tabela PESSOAS			Tabela AUTOMÓVEIS			
Nome	CPF	Telefone	Fabricante	Modelo	Ano	País
Carla Silva	618566534-47	não há	Fiat	Uno	2022	Itália
Alberto Santos	232588617-49	(57) 23832-1613	Renault	Sandero	2021	França
Mariana	454842142-46	(64) 25721-7650	Peugeot	405	2022	França
Diogo Pereira Silva	215605037-69	(16) 65016-0452				

Figura 1.03 – Exemplos de relações



A partir deste ponto, **vamos utilizar os termos “relação” e “tabela” como sinônimos**. Pode-se dizer também que tabelas (ou o formato

tabular de linhas e colunas) são uma forma de abstração adequada para representar o conceito matemático de relações. É fato que, na prática dos sistemas computacionais relacionais, a grande maioria dos desenvolvedores costuma utilizar muito mais o termo tabela do que o termo relação, pois a associação visual de certa forma facilita a comunicação. Porém, veremos que relações são tabelas com propriedades bem específicas.

Cada linha ou tupla de uma relação representa fatos que desejamos guardar ou registrar no banco de dados. As linhas, ao contrário de tabelas ou planilhas em geral, não têm identificação externa ou numeração. **Tuplas numa relação não são ordenadas**, mesmo que pareçam estar em alguma ordem quando apresentadas em sua abstração tabular. Consequentemente, não se pode falar de linha de número 1 ou 3, mesmo que possa parecer, visualmente, que uma das linhas é a primeira e a outra a terceira. É verdade que, se tivermos acesso aos detalhes internos da implementação de uma tabela, saberemos que existe alguma ordem no seu armazenamento interno. Entretanto, no mundo relacional, **qualquer tupla numa tabela pode ser considerada como a “primeira” tupla**, e seja qual for a ordem o significado daquela tabela sempre será o mesmo. Na figura abaixo, temos duas representações da mesma tabela PESSOAS. As tuplas são idênticas, com quatro linhas em ordem diferente, mas contendo essencialmente a mesma informação em cada uma delas.



Figura 1.04 – Relações não têm ordem



Os atributos de uma relação, ou as colunas de uma tabela, também não são identificados por sua ordem ou posição. É possível identificar por seu nome (de atributo ou coluna) ao qual normalmente há uma semântica associada, além de um domínio de valores válidos. A ordem das colunas é arbitrária, isto é, **há uma ordem relativa entre as colunas de uma relação, mas podemos trocar as colunas de posição sem prejuízo à interpretação de cada um dos seus elementos** (linhas ou tuplas).

Fique ligado

A ordenação dos atributos numa relação $R(A_1, A_2, \dots, A_n)$ com N atributos pode ser qualquer, mas deve ser definida, e os valores em cada tupla $t = \langle v_1, v_2, \dots, v_n \rangle$, onde $v_1 \in A_1, \dots, v_n \in A_n$ são considerados ordenados.

dados nas duas tabelas que exibem a instância corrente para efeito de consultas.

Tabela GOSTAM		Tabela VENDEM	
Pessoa	Cerveja	Bar	Cerveja
Ana	Ipa	Depósito	Leffe
Alberto	Leffe	Pires	Jupiler
Alberto	Ipa	Padoca	Ipa
Antonio	Jupiler	Pires	Gavea
Alberto	Gavea	Padoca	Copa
Carla	Copa	Depósito	Gavea
Carla	Jupiler	Padoca	Patricia
Antonio	Leffe	Padoca	Gavea

Figura 1.10A – Esquema relacional do BD cervejas



Normalmente, esquemas mudam menos do que instâncias. Pode acontecer de querermos acrescentar ou retirar uma coluna de uma tabela, ou ainda acrescentar novas tabelas por completo, mas mesmo assim não seria algo frequente. Já as instâncias, como representam os dados válidos, podem mudar diariamente, ou até mesmo várias vezes ao dia. Numa universidade, por exemplo, os alunos de uma turma mudam a cada trimestre ou semestre, mas o esquema da tabela de nome Alunos se mantém inalterado. No mercado automobilístico, novos carros surgem e outros tradicionais são descontinuados, e preços de tabela também mudam com o tempo, trazendo mudanças para a instância do BD Carros. Analogamente, os bares estão sempre com novidades no cardápio de cerveja, que, por sua vez, vão trazer novos apreciadores e, conseqüentemente, alterar a instância do BD Cervejas.

Manuseio de dados no modelo relacional

A preocupação inicial de Ted Codd quando propôs o modelo de dados relacional dizia respeito ao uso de relações como a estrutura de representação dos dados em alto nível, sem que fosse necessário ao usuário do sistema saber como relações seriam implementadas na prática. Porém, para que se mostrassem úteis, relações teriam que ser manuseadas de tal forma que consultas e operações sobre os dados permitissem responder perguntas relevantes e significativas sobre eles.

Assim, considerando relações como os únicos objetos existentes para representar os dados num sistema de bancos de dados relacional, mostra-se necessário dispor de uma Linguagem de Manuseio de Dados (LMD, ou *Data Manipulation Language* – DML) fechada sobre relações. Isto é, uma linguagem relacional, em que uma ou mais relações correspondem à entrada (*input*) e a saída (*output*) seja uma nova relação.

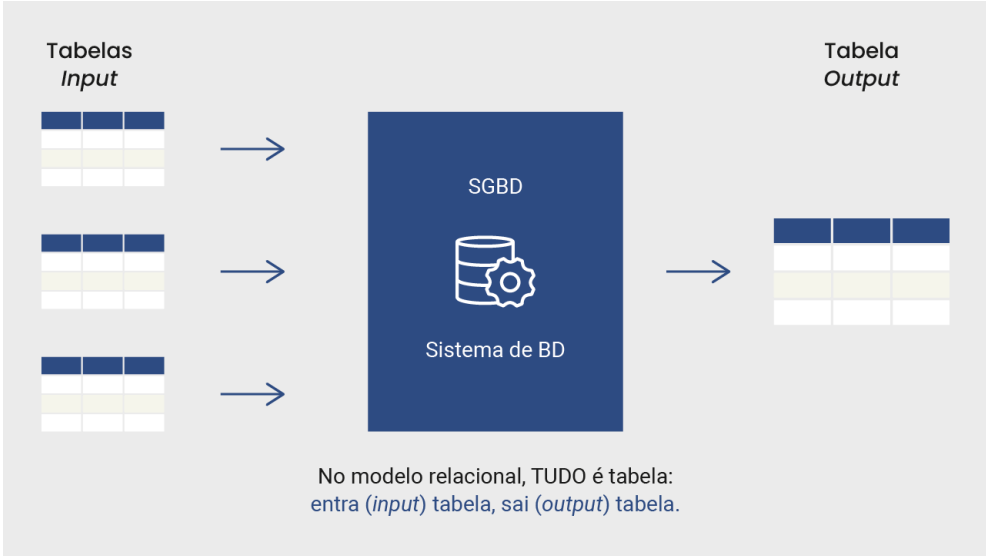


Figura 1.10B – Linguagens fechadas no mundo relacional



Além disso, é necessário também definir uma linguagem para lidar com a estrutura das relações em si, neste caso uma Linguagem de Definição de Dados (LDD, ou *Data Definition Language* – DDL).

O modelo relacional originalmente se baseou em duas LMD ditas puramente relacionais: **cálculo relacional e álgebra relacional**. O cálculo, por sua vez, se apresentou em duas versões: cálculo relacional de tuplas (*tuple relational calculus*) e o cálculo relacional de domínio (*domain relational calculus*). A linguagem Álgebra Relacional contém cinco operadores básicos: União, Diferença e Produto Cartesiano, que são operadores de manuseio de relações oriundos da teoria de conjuntos; e operadores próprios de relação, chamados de Projeção (*Project*) e Seleção (*Select*). Em termos de poder de expressão computacional, ambas as linguagens são equivalentes.

Fique ligado

Na prática dos sistemas de bancos de dados relacionais, outra linguagem se tornou padrão de fato ao longo dos anos, e assim é até hoje: ***Structured Query Language*, ou simplesmente SQL**. Trata-se de uma linguagem simples, mas bastante poderosa, baseada no formalismo do cálculo relacional e na lógica de primeira ordem (*first-order logic*, ou FO).

Embora a linguagem SQL faça referência apenas à fração de consultas (*query language*) num banco de dados relacional, ela se tornou padrão de linguagem em sistemas relacionais para todo tipo de manuseio, tanto LDD quanto LMD. Para não nos estendermos nas especificidades de cada linguagem, vamos aqui estudar apenas a linguagem SQL mesmo. As referências bibliográficas contêm os conteúdos necessários para aqueles que desejarem conhecer as demais linguagens, essencialmente teóricas, mas que não deixam de ser relevantes para a compreensão do modelo relacional.

O acesso e manipulação de dados através da linguagem SQL é realizado de forma **declarativa**. Ou seja, quando realizamos uma consulta com SQL, não precisamos **especificar como** o gerenciador irá obter os dados desejados. De fato, precisamos apenas **definir o que** desejamos que seja retornado a partir da consulta.

Na figura abaixo, ilustra-se de maneira simplificada o caminho percorrido por uma expressão de consulta na linguagem SQL internamente a um SGBD até retornar a resposta ao usuário. O motor

de execução relacional executa a sequência de operações correspondente, seguindo um plano de execução de consulta (*Query Execution Plan* – QEP) proposto pelo Otimizador do SGBD. Além da escolha dos operadores mais apropriados para realizar a consulta, o Otimizador gera um QEP utilizando estatísticas presentes nos metadados do banco.

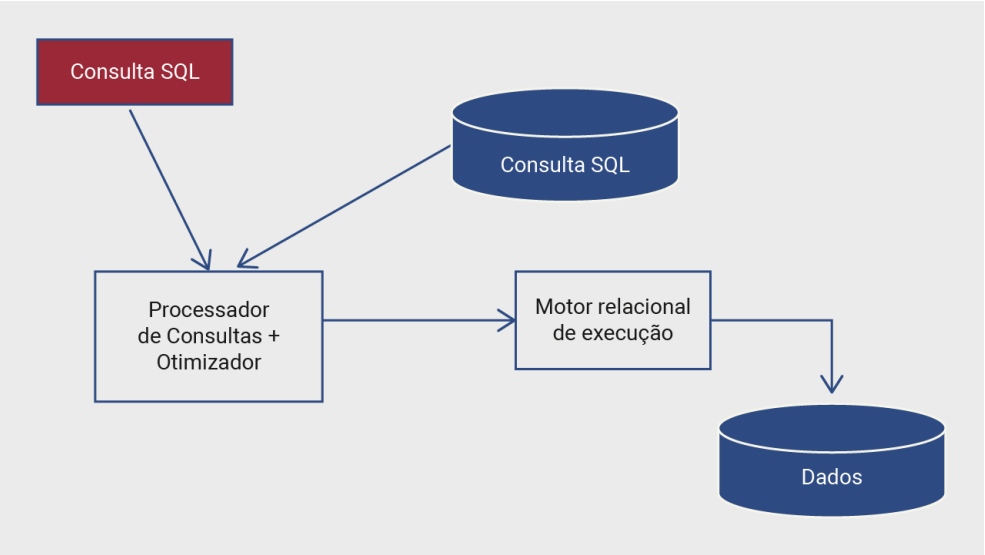


Figura 1.11 – Componentes de um SGBD e consulta SQL



Na presença de múltiplas tabelas, com possivelmente múltiplas colunas cada, de fato seria muito difícil para um programador ou desenvolvedor propor ele mesmo “o caminho das pedras” para chegar aos dados desejados de maneira a cumprir os requisitos esperados de eficácia e eficiência. A facilidade de expressar consultas de maneira declarativa é compensada — de maneira interna ao SGBD e transparente aos usuários — com um Processador e Otimizador de Consultas que garante que a tarefa será executada como esperado e no menor tempo possível.

SQL: a linguagem relacional-padrão

A linguagem SQL é uma linguagem de consultas simples, com sintaxe reduzida, e que será, nesta aula, apresentada com suas cláusulas mais importantes e utilizadas na prática. Cabe lembrar que a SQL é limitada em seu poder de expressão, não sendo uma linguagem de programação completa (não pode simular uma máquina de Turing universal), e sim voltada somente para consultas no escopo do modelo de dados relacional.

Nestes exemplos, utilizaremos alternadamente o esquema relacional Carros e o banco de dados de Cervejas para ajudar a explicar a sintaxe e semântica da linguagem.

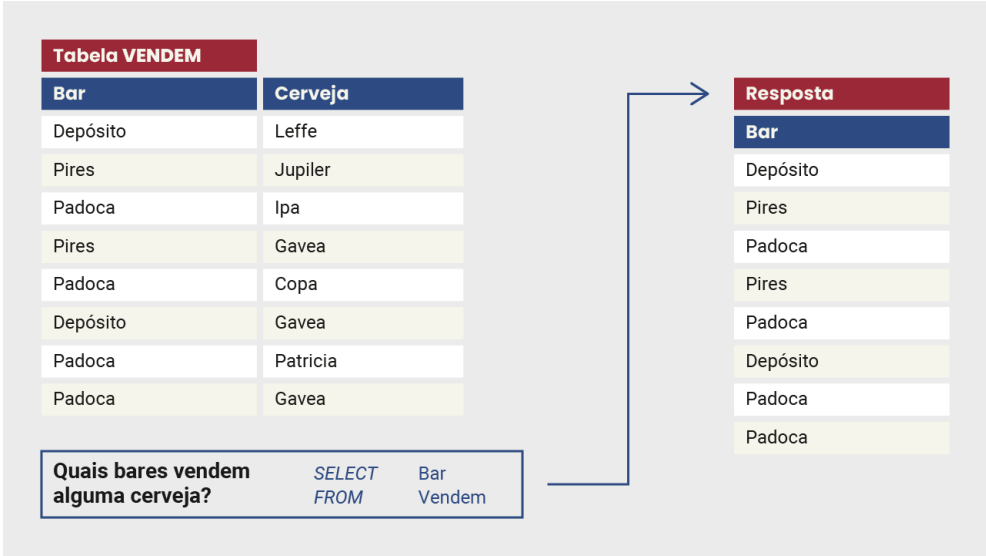


Figura 1.12A – SQL básico - BD cervejas



Considerando então a linguagem SQL na sua fração voltada para consultas, é preciso citar logo de início a cláusula *SELECT*, que é obrigatória, devendo vir acompanhada de uma lista com uma ou mais colunas (separadas por vírgulas) da tabela cujos valores se deseja visualizar como resultado. Por ser a 1ª cláusula da SQL, muitos profissionais da área, quando querem que alguém defina e execute uma consulta, usam a expressão “faz um *select*”.

Após a lista de colunas, deve-se indicar a(s) tabela(s) que contêm os dados que buscamos. Para isso, inclui-se a cláusula *FROM*, também obrigatória, seguida do nome da(s) tabela(s). Nesta aula, com o intuito de manter um nível introdutório, vamos trabalhar com apenas uma tabela na cláusula *FROM*, o que já permite realizarmos consultas interessantes e relevantes. Em outra aula, serão exemplificados casos com duas ou mais tabelas e as consequências disso nas expressões em SQL.

O comando SQL apresentado na Figura 1.12A permite listar todos os nomes de bares que vendem alguma cerveja. Nesse comando, a coluna Bar aparece logo após a palavra *SELECT*, enquanto o nome da tabela é especificado (Vendem) após a palavra *FROM*. Expressões em SQL usam como separadores de cláusulas, nomes de tabelas e colunas apenas espaços em branco ou vírgulas. Assim, a cláusula *SELECT* está em linha diferente da cláusula *FROM* por questões didáticas apenas.

Considerando outro esquema relacional diferente (banco de dados Carros), temos na figura abaixo um exemplo de consulta simples usando um *SELECT* e *FROM*, ainda sobre uma única tabela. Observe que selecionamos duas colunas da tabela Consumidores neste caso.



Figura 1.12B – SQL básico - BD carros



Podemos identificar que o resultado de uma consulta possui somente as colunas especificadas no comando (nesse caso, Cidade e UF). Além disso, essas colunas aparecem na ordem em que foram especificadas na consulta.

Sabemos que por definição as relações persistidas no banco de dados não têm tuplas repetidas. Isto pode ser controlado por meio de regras explícitas de controle de integridade que o SGBD impõe. Entretanto, ao realizar uma consulta, mesmo sabendo que as tabelas utilizadas como entrada não têm duplicatas, o mesmo não se pode garantir quanto às tabelas obtidas na saída. Além disso, são relações temporárias e não persistidas, sem controle de integridade, em particular, de unicidade de tuplas.

Para isso, a sintaxe da linguagem SQL inclui uma cláusula adicional *DISTINCT* que faz com que não tenhamos tuplas na tabela-resposta que sejam repetidas. Temos na figura abaixo um exemplo que ilustra esta situação e resolve a exibição de duplicatas.

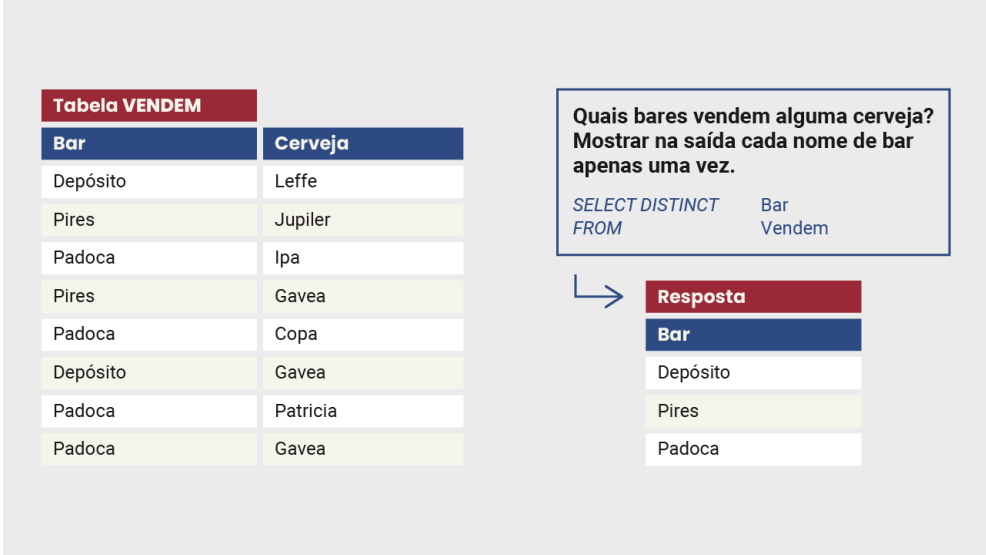


Figura 1.12C – SQL com distinct - BD cervejas



Se no esquema relacional do banco de dados tivermos uma tabela com muitas colunas, e se em alguma consulta quisermos ver todas estas colunas na saída, pode ser custoso, e sujeito a erros, precisar listá-las junto da cláusula *SELECT*. Assim, a sintaxe da linguagem SQL inclui uma maneira de simplificar esta expressão, usando o asterisco, como pode-se ver abaixo.



A figura abaixo traz uma situação destas: gostaríamos de expressar uma consulta em SQL que mostre pessoas na tabela Gostam, mas somente aquelas que gostam da cerveja Leffe. Percorrendo-se a tabela Gostam tupla por tupla, aparecerão na saída apenas aquelas tuplas cuja coluna Cerveja contenha o valor igual a Leffe. Neste exemplo particular, a cláusula *SELECT* restringe também a resposta a apenas uma das colunas, o nome das pessoas que apreciam a Leffe.

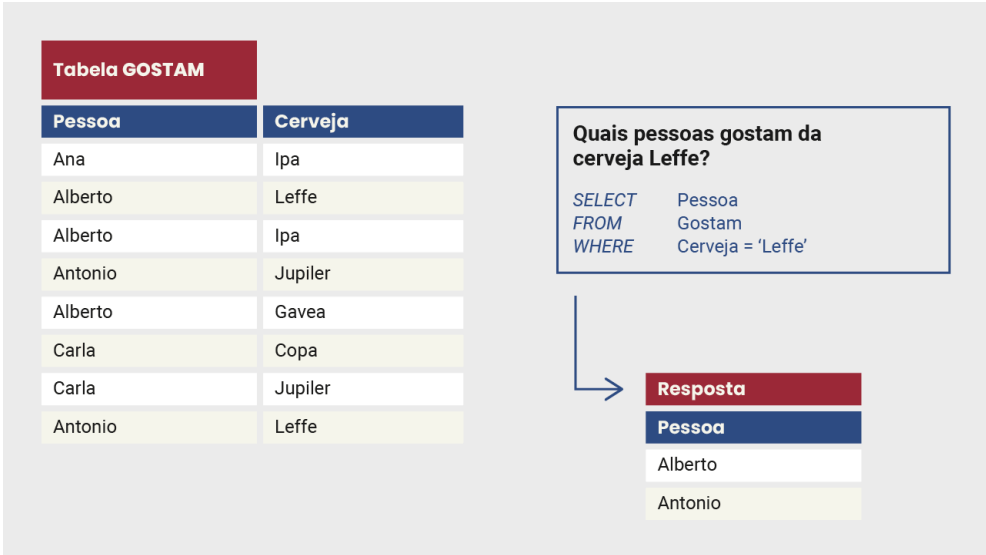


Figura 1.15A – Predicados restringindo tuplas - BD cervejas

Estes predicados relacionais da cláusula *WHERE* contêm expressões lógicas que serão validadas como verdadeiras ou falsas para cada instância particular do banco de dados. Naturalmente, pode ser útil utilizar conjunções (*AND*) ou disjunções (*OR*) em algumas expressões na cláusula *WHERE*.

Na figura abaixo, temos duas situações diferentes: no primeiro caso desejamos obter o nome das pessoas que gostam não somente de Leffe como também de Ipa. Podemos estender a expressão da figura com a disjunção *OR* e, assim, incluiríamos na resposta as pessoas que gostam tanto de Leffe quanto de Ipa. No segundo caso, com o uso da conjunção *AND*, temos como resposta uma tabela vazia pois nenhuma pessoa pode se chamar Ana e Carla ao mesmo tempo.

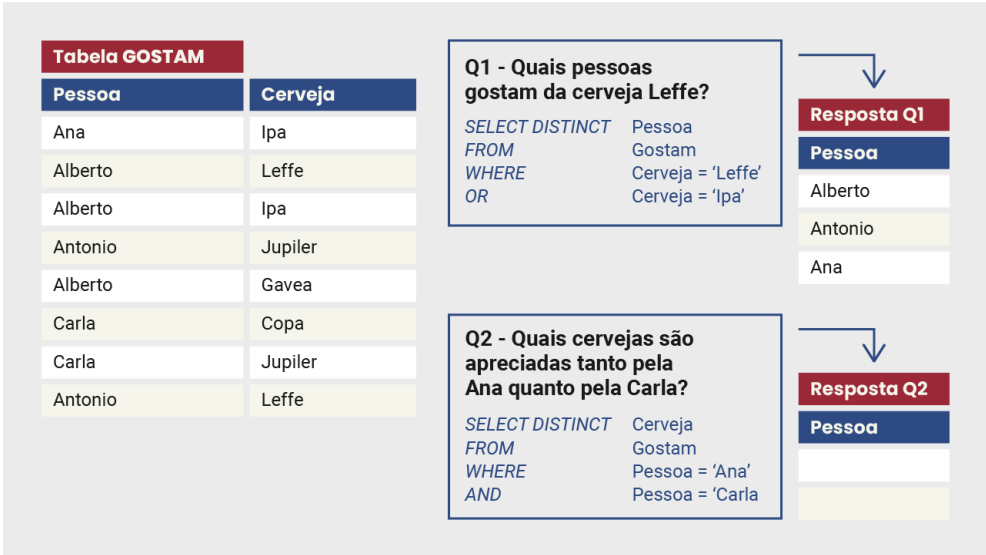


Figura 1.15B – Predicados com conjunções ou disjunções

Assim como nas linguagens de programação tradicionais, precisamos ter muito cuidado na hora de expressar consultas com operadores lógicos *AND* ou *OR*, pois nem sempre a interpretação é clara aos usuários e desenvolvedores. Ainda na figura acima, temos um exemplo de expressão usando conjunção que retorna uma relação (ou tabela) vazia. De fato, não há valores para marcas de cerveja que tanto a Ana (cerveja Ipa) quanto a Carla (cervejas Copa e Jupiler) gostam.

As expressões da cláusula *WHERE* envolvem de maneira geral comparações de atributos com valores, ou ainda atributos com atributos. Os símbolos de operadores de comparação como igual (=), maior (>) e menor (<), ou diferente (<> ou !=), podem ser usados nos predicados que restringem as tuplas que atendem a lógica da consulta.

Existe uma exceção do uso destes símbolos de comparação para quando se consideram os nulos. Se de alguma forma a consulta busca comparar colunas com valor *NULL*, deve-se utilizar *IS NULL* ou *IS NOT NULL*. São fornecidos dois exemplos distintos para estas situações abaixo.

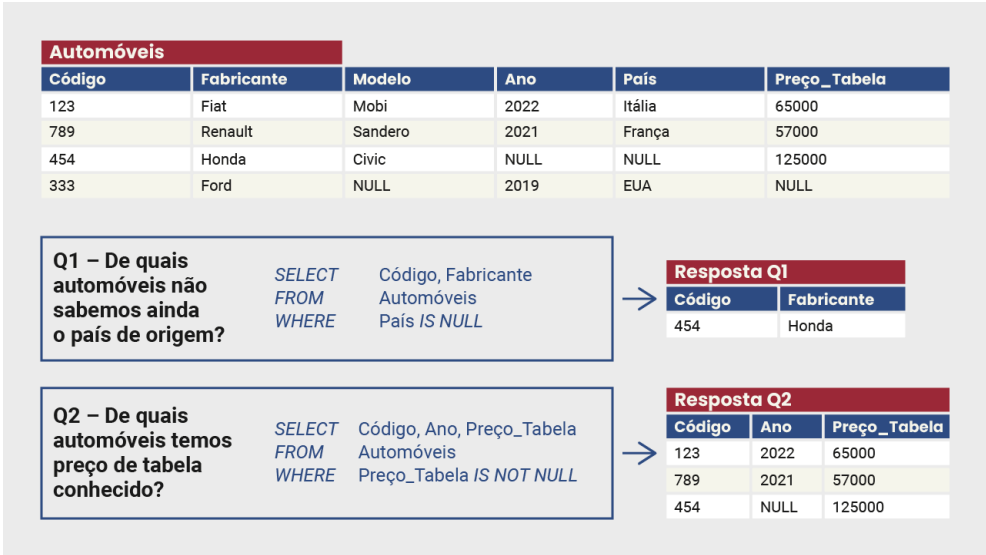


Figura 1.16 – Consultas envolvendo valores NULOS



Para concluir esta parte introdutória da linguagem SQL, precisamos explicar o importante conceito de **subconsultas aninhadas (nested subqueries)**. Como o próprio nome dá a entender, trata-se de definir consultas em SQL que contêm outras consultas SQL na mesma expressão. A figura abaixo traz um exemplo bem interessante para subconsultas: pede-se para utilizar a linguagem SQL com o objetivo de exibir na saída as cervejas que são vendidas em algum bar, mas que não as encontramos na tabela Gostam, isto é, que não são do gosto de nenhuma pessoa.

Essa operação nas linguagens relacionais é conhecida como DIFERENÇA relacional, também presente na teoria de conjuntos. Na prática, queremos saber quais cervejas estão presentes em certo conjunto (ou relação), mas não estão presentes em outro conjunto. No exemplo em particular, esta consulta permite identificar que a cerveja de marca Patrícia não é apreciada por ninguém nesta instância do banco de dados de Cervejas.

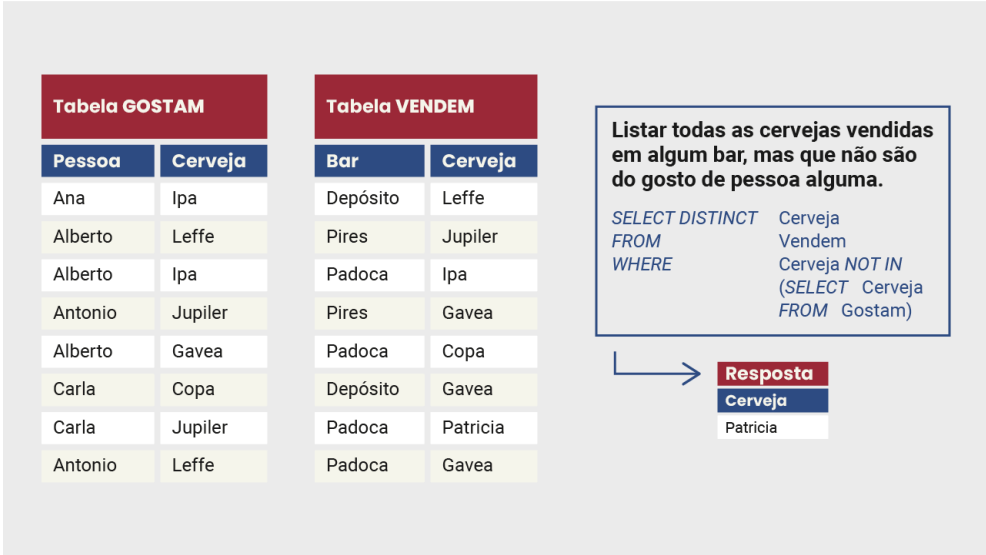


Figura 1.17A – Diferença relacional com NOT IN



Em termos sintáticos, usamos os termos *IN* e *NOT IN* para teste de pertinência de elementos aos conjuntos ou tabelas, isto é, se uma tupla pertence ou não a uma determinada relação. Assim, primeiramente identificamos quais cervejas estão presentes na tabela Gostam. Em seguida, comparamos quais cervejas na tabela Vendem não pertencem ao conjunto de cervejas presentes na tabela Gostam, para responder adequadamente à consulta.

A figura abaixo contém um exemplo de consulta SQL que utiliza o operador *IN* para explicitar os valores de comparação utilizados em consultas com subconsultas aninhadas, em que se deseja verificar se elementos são membros ou pertencem a determinadas tabelas (ou conjuntos). Trata-se do mesmo exemplo da figura **Predicados com conjunções ou disjunções**, mas com sintaxe simplificada, já que sabemos quais valores queremos comparar. Ou seja, nem sempre é necessário determinar os valores da tabela correspondente à subconsulta antes de executar a consulta completa. Às vezes, já conseguimos explicitar os valores de comparação desejados.

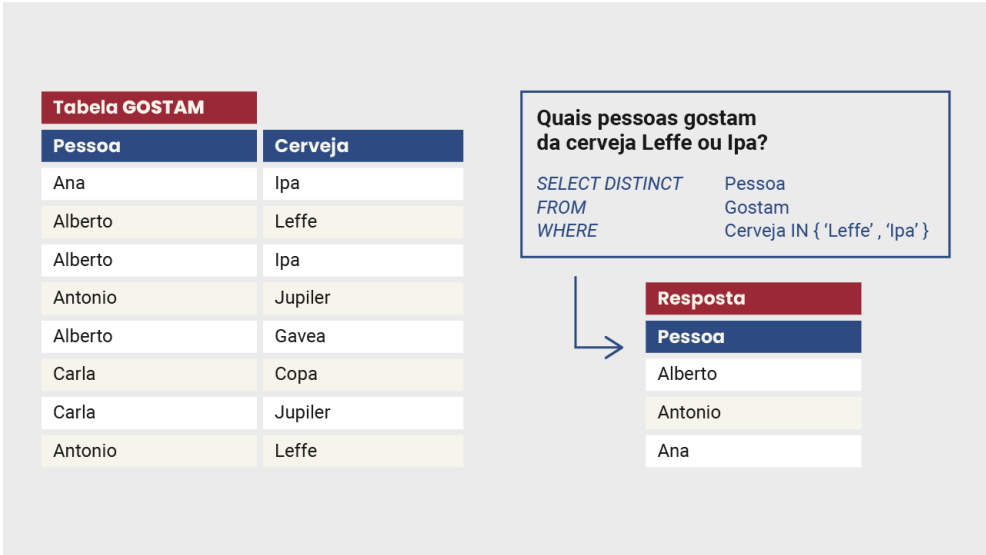


Figura 1.17B – Conjuntos explícitos



Cabe observar que a linguagem SQL permite representar a diferença de conjuntos de outras maneiras, como, por exemplo, com o operador *EXCEPT*. Outros operadores de conjuntos, como *UNION*, também fazem parte da linguagem SQL, mas estão fora do escopo do material desta aula em particular.

Premissas e limites do modelo relacional

A linguagem SQL tem uma série de características interessantes e fundamentais para que possa ser utilizada com correção e robustez. Para isso, convidamos o aluno a assistir este vídeo que discute alguns temas bem relevantes, como a hipótese do mundo fechado, a orientação aos conjuntos e a importância de não termos elementos repetidos nas relações no modelo.



Restrições de integridade em bancos de dados

Sabe-se que é possível determinar o modelo lógico de um banco de dados em função das duas partes do formalismo matemático escolhido: **a estrutura dos dados e a forma de manuseio deles**. No caso do modelo de dados relacional, temos como estrutura as relações, ou tabelas, e como manuseio temos as linguagens

relacionais, em particular a linguagem SQL, padrão de mercado e bastante utilizada na prática. Já para determinar um esquema de um banco de dados relacional em particular, precisamos detalhar sua estrutura, a saber, as tabelas que pertencem ao banco de dados com as respectivas colunas.

Todavia, como não faz sentido termos dados incorretos ou inválidos, é importante definir algumas regras que determinem quais dados podem ser aceitos para popular as tabelas e quais não podemos aceitar, de forma que o banco de dados seja efetivamente útil para os sistemas que dele fazem uso.

Estas regras são conhecidas como **restrições de integridade** (*integrity constraints*) e, dada a importância que têm, na prática acabam fazendo parte da definição do esquema relacional de um banco de dados particular. Para qualquer modelo de dados, podemos especificar restrições chamadas estruturais — referentes ao modelo escolhido — e as restrições específicas da aplicação, também conhecidas como **regras de negócio** (*business rules*), que complementam a especificação da semântica que se espera dos dados armazenados no banco.

Fique ligado

No caso do modelo relacional, temos restrições estruturais que dizem respeito ao fato de estarmos usando relações como estrutura de representação dos dados:

1 - restrição de chave;

2 - restrição de entidade;

3 - restrição de domínio;

4 - restrição de integridade referencial.

A ideia básica é definir conceitos e estipular condições que possam ser implementadas e controladas automaticamente pelo SGBD relacional escolhido. Desta maneira, teremos uma gestão eficaz dos dados no banco, garantindo que não serão obtidas informações inesperadas que comprometam os sistemas que se servem daqueles dados.

Nesta aula, veremos inicialmente as restrições **1 (chave)** e **4 (integridade referencial)**. Na próxima aula, as demais restrições, inclusive as regras de negócio, serão abordadas em mais detalhes.

Controlando tuplas repetidas com chaves primárias

Sabemos que relações não têm elementos repetidos, assim como os conjuntos. Caso contrário, não teríamos como distinguir uma linha da outra, pois os valores em todas as colunas de uma linha seriam exatamente iguais ao da outra linha.

Fique ligado

A restrição de **integridade de chave** busca definir uma maneira de controlar esta condição de unicidade de tuplas em qualquer das tabelas de um esquema relacional. Na prática, temos a necessidade de diferenciar uma tupla da outra, pois estes representam os elementos de dados que serão utilizados no sistema computacional. Como não há identificador de linhas, cada uma das linhas de uma tabela deve conter pelo menos uma coluna, ou conjunto de colunas, que a identifica unicamente, contendo valores distintos. A isso chamamos de chave (*key*, em inglês) da tabela a coluna, ou conjunto de colunas, que permite esta identificação das tuplas em uma relação. Muitas vezes, temos colunas, ou conjunto de colunas, que naturalmente assumem valores diferentes para cada tupla e, assim, são consideradas **chaves candidatas**.

Suponhamos uma relação chamada Pessoa, contemplando atributos como nome, CPF, RG e UF (unidade federativa), data de nascimento, entre outros. Poderíamos ter uma chave candidata K1 = {CPF} e outra chave K2 = {RG, UF}. Observe que a chave K2 também é dita composta, pois contém mais de um atributo. Ademais, a lei brasileira permite números de RG iguais em estados diferentes, por isso K2 deve ser composta.

Um leitor mais atento poderia sugerir como identificador das linhas numa tabela a “chave” K3 = {CPF, Nome}. De fato, não há duas linhas na tabela com valores iguais para este par de colunas. Porém, sabemos que não há duas pessoas com o mesmo CPF, e temos em K3 um atributo adicional e desnecessário para identificação. Na teoria do relacional, K3 é dita uma super-chave (*super-key*, ou SK) da tabela. É um conceito que diz respeito a qualquer conjunto de colunas numa tabela que identifique unicamente suas linhas.

A diferença de uma chave para uma super-chave é que a primeira é dita *minimal*, no sentido de que a retirada de qualquer atributo do conjunto faz com que o mesmo deixe de ter a propriedade de identificar tuplas!

Com dois ou mais atributos, é preciso verificar se não há atributos adicionais sem necessidade. A super-chave K3 segue identificando linhas se retirarmos o atributo Nome do conjunto, explicitando o fato de que não se trata de um conjunto *minimal*; logo, não é chave candidata. Qualquer conjunto com apenas um atributo que identifique unicamente tuplas numa tabela pode ser chamado de chave (candidata) daquela tabela. O conjunto K2 é chave, pois, mesmo composta de dois atributos, tem a propriedade de ser *minimal*: somente com RG ou somente com UF não é possível identificar linhas na tabela.

Na teoria do modelo relacional, define-se que cada tabela pode ter apenas uma chave e esta pode ser definida arbitrariamente, sem prejuízo à definição de chave ou à propriedade de identificação das linhas. **Esta chave única de cada tabela é chamada de chave primária (*primary key*, ou PK), e as demais chaves seguem como candidatas**. Se a chave K1 tiver sido escolhida como PK, a chave K2 segue como candidata, e vice-versa. Note que não existe a noção de

chave secundária no modelo relacional: temos apenas chaves primárias ou chaves candidatas. Além disso, se a PK tem dois atributos, diz-se apenas que a PK é composta, mas não deixa de ser única — não são “duas PKs”, como muitos interpretam erroneamente.

Por definição, todos os atributos de uma relação (qualquer!) formam uma SK, já que não teremos tuplas com valores iguais ou repetidas. Quando não temos naturalmente subconjuntos do conjunto completo de atributos da tabela que permitam identificar as linhas de uma tabela, recorreremos ao artifício de considerar uma **chave artificial** (*surrogate*) para identificar as tuplas. É o caso dos identificadores como CPF, de pessoas ou contribuintes, ou mesmo o atributo Matrícula de alunos em universidades ou escolas. Como existem homônimos possíveis, busca-se simplificar a definição da PK com um único atributo que, por construção, não se repete para qualquer indivíduo. O mesmo vale para empresas (CNPJ) ou mesmo códigos de estados ou países para sistemas de telefonia (DDD ou DDI, respectivamente). Alguns desenvolvedores sugerem colunas adicionais na tabela com contadores (ou seriais) para servirem como PK. Entretanto, diferentemente dos demais exemplos que têm interpretação apropriada às aplicações particulares, atributos com números sequenciais são artifícios utilizados para, de alguma forma, referenciar tuplas como se fossem registros ordenados em arquivos, o que não é o caso.

Utilizar um conjunto muito grande de atributos como PK de uma tabela pode implicar em problemas de desempenho no sistema de banco de dados, pois o processamento que permite identificar tuplas será mais demorado. Entretanto, atributos artificiais não têm semântica própria: entendemos perfeitamente que Uruguai é um país da América do Sul, mas não é imediato associar que 598 é um código de telefonia daquele país.

Sabe-se que, na prática, em função das características particulares do sistema de banco de dados relacional que será desenvolvido, a escolha da chave primária pode não ser tão arbitrária assim. Neste caso particular de identificação de pessoas, a chave K1 poderia ser considerada primária para bancos de dados voltados mais às aplicações financeiras. Já no caso de problemas aplicados com identificação civil, a chave K2 seria mais adequada.

Por todos esses motivos relatados, o modelo relacional também é conhecido como orientado aos valores (*value-oriented*) e não orientado a linhas, tuplas ou mesmo registros, como analogia visual aos arquivos considerados pelas linguagens de programação, que são ditos orientados aos registros (*record-oriented*) ou mesmo aos objetos (*object-oriented*) com seus identificadores.

Integridade referencial e chaves estrangeiras

Existe uma situação particular de controle de integridade que procura estabelecer um relacionamento entre colunas de tabelas diferentes, utilizando como domínio de valores válidos para uma coluna C1 na tabela T1 os valores que estão presentes na coluna C2 da tabela T2. Como exemplo, podemos citar que os valores dos estados (coluna UF) na tabela de Pessoas deve ser igual aos valores de UF presentes numa tabela contendo apenas os estados brasileiros. Dessa maneira, garantiríamos que os códigos de UF

utilizados seriam todos válidos. Analogamente, podemos citar também que os valores para marcas de cerveja presentes nas tabelas Gostam ou Vendem também seriam apenas os valores existentes numa tabela Origem, que lista marcas de cerveja e seus país de fabricação original. Ou ainda seria uma maneira de garantir que o valor Astrologia não seria aceito como o curso de um aluno da PUC-Rio, pois não existe este curso na tabela de cursos da instituição presentes no banco de dados.

Fique ligado

Esta restrição é conhecida como **integridade referencial** exatamente por envolver valores em determinada tabela que fazem referência a valores em outra tabela. De maneira um pouco mais formal, e buscando evitar algumas possíveis inconsistências, a restrição de integridade referencial diz que os valores da coluna C1 da tabela T1 devem referenciar valores em uma coluna C2 que é necessariamente chave primária (PK) da tabela T2, ou então podem assumir valor *NULL*. A coluna C1 é definida como chave estrangeira (*foreign key*, ou FK) da tabela T1.

Cabe observar que esta definição de restrição de integridade com chaves estrangeiras referenciando chaves primárias não se aplica apenas a atributos isolados. Se a chave primária da tabela T2 for composta de um ou mais atributos, só poderemos ter chaves estrangeiras se pelo menos a quantidade de atributos for a mesma. Além disso, o domínio dos valores deve ser o mesmo para cada uma das colunas envolvidas na FK e na PK referenciada. Logo, se estamos falando da PK {RG, UF} em determinada tabela, é preciso existir um outro par de colunas {C3, C4} em alguma tabela tal que os domínios de valores em C3 e C4 sejam iguais aos domínios de RG e UF, respectivamente. A integridade referencial neste caso só seria satisfeita se valores do par {C3, C4} forem iguais a *NULL* ou, caso contrário, iguais a valores do par {RG, UF}. Por último e não menos importante: é possível, mesmo sendo mais raro, que uma FK aponte para uma PK da mesma tabela! Ou seja, não necessariamente T1 e T2 serão tabelas diferentes.

Conclusão

Sabe-se que os sistemas de bancos de dados são criados e projetados para lidar com bancos de dados com grande volume (*Very Large Databases* — VLDB) e aprende-se a lidar com estes sistemas usando linguagens como SQL declarativas, com as quais dizemos o que queremos sem entrar em detalhes de como isso será feito pelo computador. Neste momento, cabe perguntar: como saber se o resultado de uma consulta sintaticamente correta (e que, portanto, será executada!) está semanticamente correto? Se temos de fato um VLDB, é praticamente impossível validar o resultado mediante uma verificação visual ou manual.

Outra dúvida muito comum se apresenta quando estudamos os sistemas de bancos de dados relacionais. Afirma-se que as tabelas ou relações, teoricamente, seguem a teoria dos conjuntos, e não podem conter elementos repetidos. Mas na prática as tabelas que são resultados de consultas quase sempre representam os chamados multiconjuntos (*multisets*), e tuplas repetidas podem ser exibidas como saída de uma execução de consulta SQL. **De que maneira lidamos na prática com esse problema?**

Referências

Clique aqui para acessar as referências e créditos desta aula.

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, v. 13, n. 6, p. 377-387, 1970.

COSTA, R. L. C. **SQL**: guia prático. Rio de Janeiro: Editora Brasport, 2006.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de bancos de dados**: fundamentos e aplicações. 6 ed. Campinas: Pearson Universidades, 2010.

GARCIA-MOLINA, H.; ULLMAN, J. D.; WIDOM, J. **Database systems**: the complete book. Hoboken: Prentice Hall, 2008.

HEUSER, C. A. **Banco de dados relacional**. Joinville: Clube de Autores, 2019.

RAMAKRISHNAN, R.; GEHRKE, J. **Sistemas de gerenciamento de bancos de dados**. 3 ed. Porto Alegre: AMGH, 2008.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de bancos de dados**. Rio de Janeiro: Elsevier-Campus, 2012.

ULLMAN, J. D. **Principles of database and knowledgebase systems**. Ann Arbor: Computer Science Press, 1988. Vol. 1.

Banco de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock.